

Project 7 (in Java): You are to implement a binary image compression and decompression technique via quadtree. Given a binary image, your program will produce a quadtree. To verify the correctness of the quadtree that your program produced, your program will decompress the quadtree and produce a binary image. If your program works correctly, the decompressed binary image should be identical to the input image. Since quadtree needs a square image size of 2^k by 2^k , $k \geq 1$, therefore, the first step of your program does is to re-size the input image into a square image size of 2^k by 2^k .

You will be given three images (img1, img2, and img3) to test your program. img1 is a square image, img2 and img3 are not square images.

*** What do you need to do:

- a) Illustration-I: Draw/print img1 on the top of a blank page, then draw the quadtree representation of img1 below it.
Then write the color of nodes visited in a pre-order traversal of the quadtree;
and write the color of nodes visited in a post-order traversal of the quadtree.
(The color of white leaf-nodes is 0, black-leaf-nodes is 1, and internal nodes is 5.
- b) Illustration-II: Pad img2 into a 2^k by 2^k square image. Use another blank paper to do the same as in illustration I for the padded img2.
- c) Implement your program according to the specs given below.
- d) Debug your program until it passes compilation.
- e) Run and debug your program on img1 until your pre-order and post-order traversals are the same as your illustration 1.
- f) Run and debug your program on img2 until your pre-order and post-order traversals are the same as your illustration II.
- g) Run your program on img3.

=====

Include in your hard copies:

- Cover page
- Illustration-I of img1 (-1 pt if omitted.)
- Illustration-II of img2 (-1 pt if omitted.)
- Source code
- Img1 // with caption "*** Below is input img1"
- outFile1 for img1 // with caption "*** Below is outFile1 for img1"
- logFile for img1 // with caption "*** Below is logFile for img1" limit to 3 pages, if more.
- Img2 // with caption "*** Below is input img2"
- outFile1 for img2 // with caption "*** Below is outFile1 for img2"
- logFile for img2 // with caption "*** Below is logFile for img2" limit to 3 pages, if more.
- Img3 // with caption "*** Below is input img3"
- outFile1 for img3 // with caption "*** Below is outFile1 for img3"
- logFile for img3 // with caption "*** Below is logFile for img3" limit to 3 pages, if more.

Language: Java

Project Name: Binary image compression and decompression via Quad-tree

Project points: 10 pts

Due Date: Soft copy (*.zip) and hard copy (word file):

(10/10 pts): on time, 4/12/2026 Sunday before midnight, 11:59pm. 15 minutes grace period.

(-10/10 pts): non-submission, 4/12/2026 Sunday after midnight

*** All submission MUST include soft copy (*.zip) and hard copy (*.docx) in the same email attachments with correct email subject as stated in the project submission requirement; otherwise, your submission will be rejected. Email subject: (323) first name last name <Project 7: Binary image compression and decompression via Quad-tree (Java)>

*** Inside the email body includes:

- Your answer to the five questions (-2 if omitted)
- Screen recoding link. (-2 without the recording!)

*** Place your screen recording link in your project submission email body below the 5 questions.

I. Input: inFile (args [0]) a text file contains a binary image with header information. The first text-line is the header information: numRows, numCols, minVal, maxVal, follows by rows and columns of pixels, 0's and 1's. An example below is a binary image has four (4) rows and five (5) columns, minVal is 0 and maxVal is 1:

```
4 5 0 1
0 0 0 1 0
0 1 1 0 1
0 0 1 0 1
1 0 1 1 1
```

II. Outputs:

- a) outFile1 (args [1]): as specs dictates.
- b) logFile (args [2]): as specs dictates.

III. Data structure:

- QtNode class

- (int) color // Quad tree node's color. 0 and 1 for leaf-nodes and 5 for non-leaf-nodes.
- (int) upperR // the row coordinate of the upper left corner of the image area of the node is representing.
- (int) upperC // the column coordinate of the upper left corner of the image area of the node is representing.
- (int) Size // one size of the square image the node is representing.
- (QtNode) NW // NW child, initialize to null
- (QtNode) NE // NE child, initialize to null
- (QtNode) SW // SW child, initialize to null
- (QtNode) SE // SE child, initialize to null

Method:

- constructor (...): to create a QtNode.
- printQtNode (...) // If the given node is non-leaf node, print node's color, and it's 4 children's color.
// For example,
// if node is a internal node; it's NW child's color is 5, NE child's is 0, SW child's is 1, SE
// child's is 5, then print <5, 5, 0, 1, 5> →
// If node is a while-leaf node, then print <0, null, null, null, null> →
// If node is a black-leaf node, then print <1, null, null, null, null> →

- QtTree class

- (QtNode) QtRoot
- (int) numRows
- (int) numCols
- (int) minVal
- (int) maxVal
- (int) power2Size
- (int [][]) imgAry // a 2D array of size power2Size by power2Size, need to dynamically allocate at run time.
- (int [][]) newImgAry // a 2D array of size power2Size by power2Size, need to dynamically allocate at run time.
// To store the re-created new image from the Quadtree.

methods:

- constructor (...) // performs necessary allocations and initializations and others as you like.
- (int) computePower2 (...) // **see algorithm below.**
// The method determines the smallest square box size of power of 2 by power of 2
// that fits the input image. It returns the size (one side) of the power2Size
- loadImage (...) //load the input data onto imgAry, begins at (0, 0). **On your own.**
- zero2DAry (...) // set all pixels in the given 2D array to zero.
- (QtNode) buildQuadTree (...) // **see algorithm below** (a recursive algorithm).
- (int) addKidsColor (node) // add up node's 4 kids colors. **On your own.**
- (bool) isLeaf (node) // returns true if node is a leaf node (its color is 1 or 0), otherwise returns false.

- Ary2File (...) // Output content of a given 2D array of a file. **On your own.**
- preOrder (...) // **see algorithm s below.**
- postOrder (...) // **on your own.**

- fillnewImgAry (...) // **See algorithm below.** The method fills the padded portion of the newImgAry with the color
//of a given leaf node.
- getLeaf (...) // **see algorithm below.** The method traverses the Quadtree from QtRoot, when it reached a leaf
//node, it calls fillnewImgAry () to fill a portion of newImgAry with the color of the leaf.

IV. Main (...) // Use scanner for inFile for easy reading

step 0: inFile, outFile1, logFile ← open from args []

step 1: numRows, numCols, minVal, maxVal ← inFile

step 2: power2Size ← computePower2(numRows, numCols, logFile)

outFile1 ← “*** In main(), power2Size =” // print value.

step 3: imgAry ← dynamically allocate the array size of power2Size by power2Size

step 4: zero2DAry (imgAry)

step 5: newImgAry ← dynamically allocate the array size of power2Size by power2Size

step 6: zero2DAry (newImgAry)

step 7: loadImage (inFile, imgAry)

outFile1 ← “*** Below is the re-sized input image”

Ary2File (imgAry, outFile1)

step 8: QtRoot ← BuildQuadTree (imgAry, 0, 0, power2Size, logFile)

step 9: outFile1 ← “*** Below is the pre-order traversal of the Quad tree”

preOrder (QtRoot, outFile1)

step 10: outFile1 ← “*** Below is the post-order traversal of the Quad tree”

postOrder (QtRoot, outFile1)

step 11: logFile ← “*** In Main(): calling getLeaf (..) to recreate the image from quadtree”

step 12: getLeaf (QtRoot, newImgAry, logFile)

step 13: outFile1 ← “*** Below is the re-created image from Quad tree”

outFile1 ← numRows, numCols, minVal, maxVal

Ary2File (newImgAry, outFile1)

step 14: close all files

V. (QtNode) buildQuadTree (imgAry, upR, upC, size, logFile)

Step 0: logFile $\leftarrow$ “** Entering buildQuadTree: upR= upC= and size= ” // print the values of the three.

Step 1: newQtNode $\leftarrow$ get a new QtNode with: (-1, upR, upC, size, null, null, null, null)

Step 2: if size == 1 // one pixel

newQtNode’s color $\leftarrow$ imgAry [upR][upC] // one pixel, either 1 or 0

else

halfSize $\leftarrow$ size / 2

newQtNode’s NW $\leftarrow$ buildQuadTree (imgAry, upR, upC, halfSize, logFile)

newQtNode’s NE $\leftarrow$ buildQuadTree (imgAry, upR, upC + halfSize, halfSize, logFile)

newQtNode’s SW $\leftarrow$ buildQuadTree (imgAry, upR + halfSize, upC, halfSize, logFile)

newQtNode’s SE $\leftarrow$ buildQuadTree (imgAry, upR + halfSize, upC + halfSize, halfSize, logFile)

sumColor $\leftarrow$ addKidsColor (newQtNode)

logFile $\leftarrow$ “** the sumColor 4 children’s colors is” //the print sumColor

if sumColor == 0 // all 4 kids are 0, means it is a leaf-node and its color is 0.

newQtNode’s color $\leftarrow$ 0

else if sumColor == 4 // all 4 kids are 1, means it is a leaf-node and its color is 1.

newQtNode’s color $\leftarrow$ 1

else

newQtNode’s color $\leftarrow$ 5 // newQtNode is an internal node, grey.

// Why we do not set the color of the internal node to 2 or 3? Will be on exam 3.

step 3: logFile $\leftarrow$ “** Printing newQtNode below”

printQtNode (newQtNode, logFile)

step 4: logFile $\leftarrow$ “** leaving buildQuadTree”

step 5: return newQtNode

VI. (int) computePower2 (numRows, numCols, logFile)

step 0: logFile $\leftarrow$ “** Entering computePower2 method”

step 1: size $\leftarrow$ max (numRows, numCols)

step 2: power2 $\leftarrow$ 2

step 3: if size > power2

power2 $\leftarrow$ power2 * 2

step 4: repeat step 3 until size <= power2

step 5: logFile $\leftarrow$ “** Leaving computePower2 method, power2 is” // print power2

step 6: return power2

VII. getLeaf (Qt, newImgAry, logFile)

Step 0: logFile $\leftarrow$ “** Entering getLeaf method”

Step 1: If isLeaf (Qt)

fillnewImgAry (Qt, newImgAry, logFile)

else

getLeaf (Qt’s NW, newImgAry, logFile)

getLeaf (Qt’s NE, newImgAry, logFile)

getLeaf (Qt’s SW, newImgAry, logFile)

getLeaf (Qt’s SE, newImgAry, logFile)

Step 2: logFile $\leftarrow$ “** Leaving getLeaf method.”

VII. fillnewImgAry (Qt, newImgAry, logFile)

Step 0: logFile $\leftarrow$ “** Entering fillnewImgAry method”

(int) color, R, C, sz

Step 1: color $\leftarrow$ Qt’s color

R $\leftarrow$ Qt’s upperR

C $\leftarrow$ Qt’s upperC

sz $\leftarrow$ Qt’s size

Step 2: i = R

Step 3: j = C

Step 4: newImgAry [i,][j] $\leftarrow$ color

Step 5: j++

Step 6: repeat step 4 to 5 while j < C + sz

Step 7: i++

Step 8: repeat step 3 to step 7 while i < R + sz

Step 9: logFile $\leftarrow$ “**Leaving fillnewImgAry method”

VI. preOrder (Qt, outFile1)

If isLeaf (Qt)

printQtNode (Qt, outFile1)

else

printQtNode (Qt, outFile1)

preOrder (Qt’s NW)

preOrder (Qt’s NE

preOrder (Qt’s SW)

preOrder (Qt’s SE)